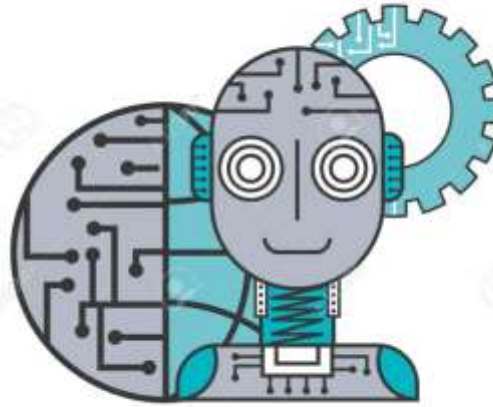




Regression Problem

Ioannis Tsioulis (MC)

Declaration

The content of the slides are modified based on:

- The book “Doing Data Science”
- The Stanford ML course <http://www.ml-class.org/>
- Some other references

Acknowledgement

Most of the slides are adapted from the lecture slides of Dr Hong Qing Yu (Harry).

Outlook

1. Linear regression

- Examples
- Math notation
- Least Mean Square function (LMS)
- Gradient Descent function

2. Logistic regression

What is linear regression

Linear regression is a commonly used statistic based predictive model to exam the relationship between scalar response with one or more independent variables.

Linear regression one type of regression analysis. Others include polynomial regression, binary regression and logistic regression and many more.

However, the linear regression model is the most basic, easy to understand and implement.

Simple example

Example 1. Overly simplistic example to start

Suppose you run a social networking site that charges a monthly subscription fee of \$25, and that this is your only source of revenue. Each month you collect data and count your number of users and total revenue. You've done this daily over the course of two years, recording it all in a spreadsheet. You could express this data as a series of points.

Here are the first four data:

$$S=\{(x, y) = (1, 25), (10, 250), (100, 2500), (200, 5000)\}$$

Exam the data in your head

$$S=\{(x, y) = (1,25), (10,250), (100,2500), (200,5000)\}$$

Some ideas?

- There's a linear pattern.
- The coefficient relating x and y is 25.
- It seems deterministic.



UNIVERSITY OF
DERBY



derby.ac.uk

Let's plot the data

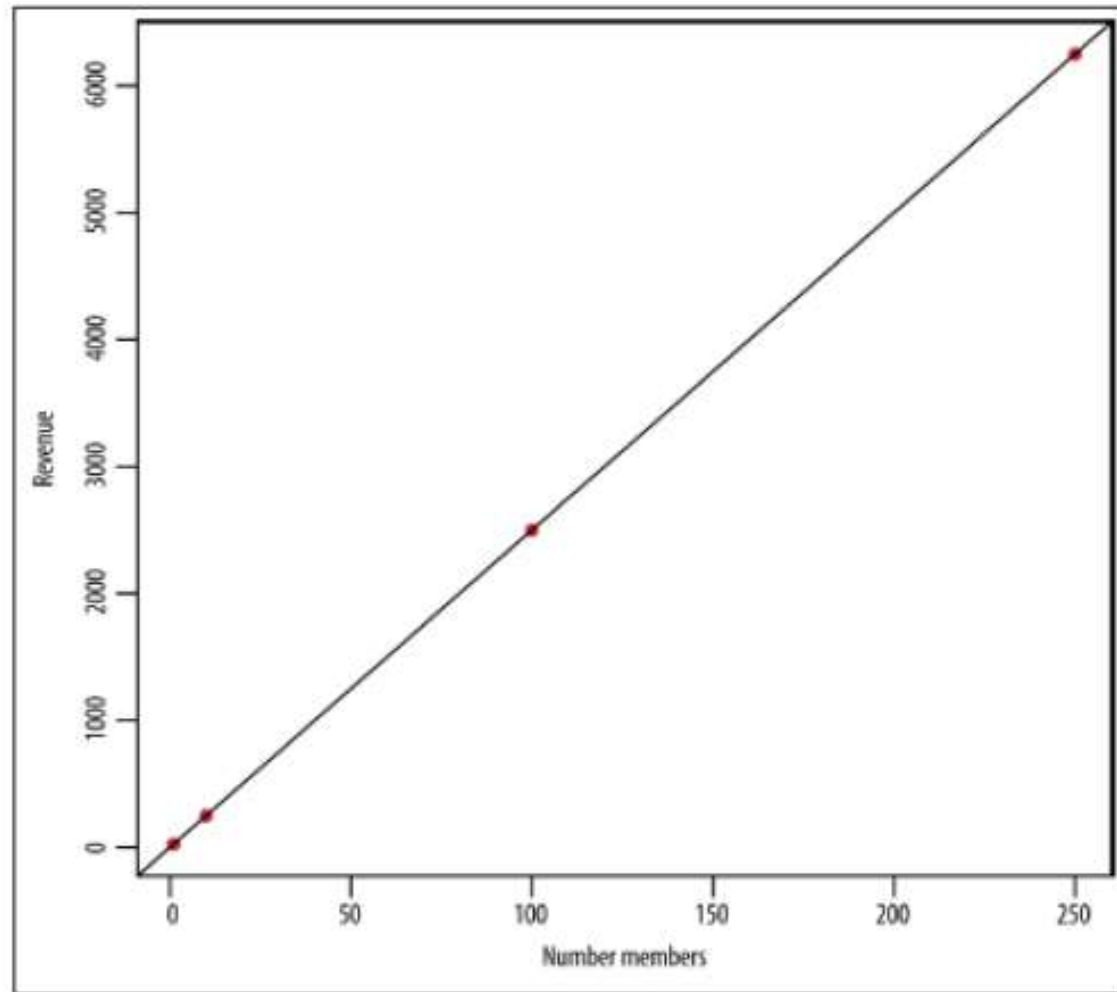


Figure 3-1. An obvious linear pattern

The figure adapt from book of “Doing Data Science” Chapter 3

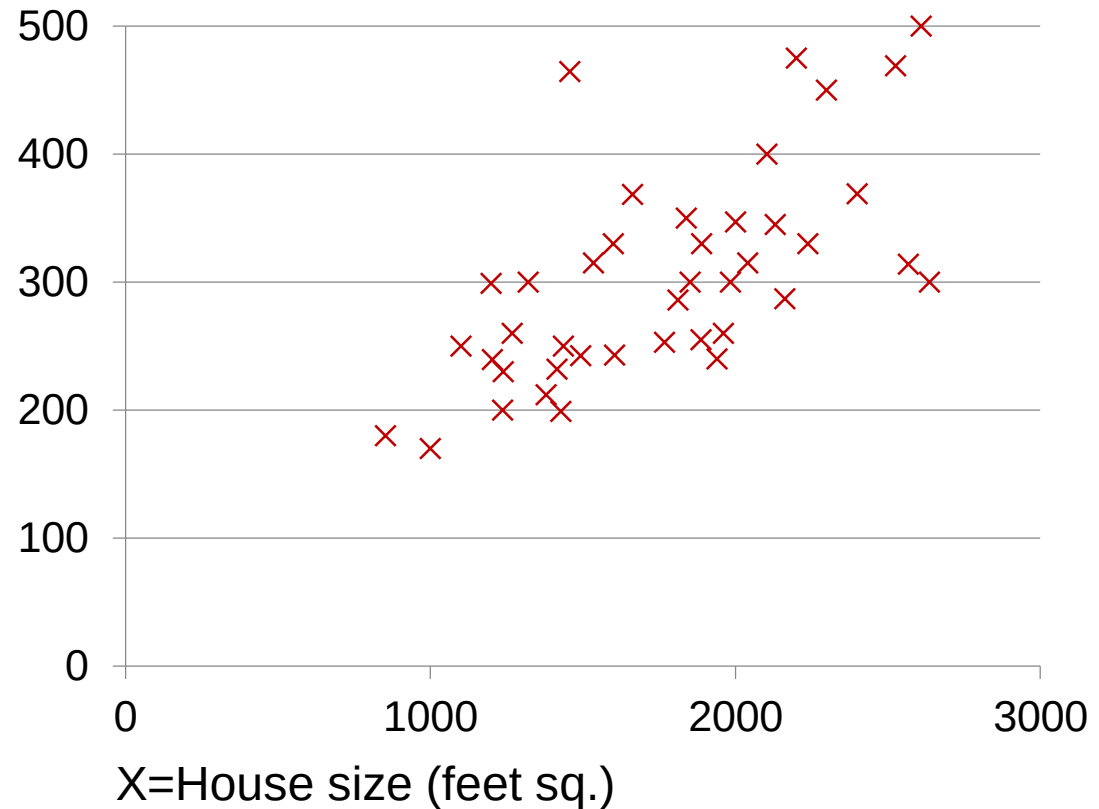
More realistic example

Example 2: housing prices

Y=House price
(\$1000)

Supervised Learning

Given the “right answer” for each example in the data.
(given a size of a house, get a price prediction)



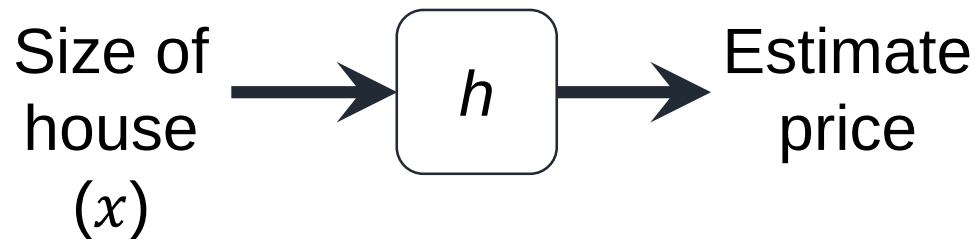
Regression Problem

Predict real-valued output

Mathematic notations

Before continuing, some notations need to be introduced

Hypothesis $h_{\theta}(x)$, where θ is the parameters which can be tuned to find the best solution for the hypothesis. The x is the input variable (features) of the hypothesis.



Linear Hypothesis:

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

Mathematic notations

Some other notations also use simplified version

$$y = \beta_0 + \beta_1 x$$

Each step of the prediction output use $\hat{y}$ instead of $h_{\theta}(x)$

	Size in feet ² (x)	Price (\$) in 1000's (y)
1	2104	460
	1416	232
	1534	315
	852	178
	...	...
m		

We use “**m**” to denote the number of training examples

Mathematic notations

If we have more than one variables, then the hypothesis becomes to:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2$$

Using **n** to denote the # variables

	Size in feet ² (x_1)	# Rooms (x_2)	Price (\$) in 1000's (y)
1	2104	6	460
	1416	4	232
	1534	5	315
	852	3	178
m	...		...

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots = \sum_{j=0}^n \theta_j x_j^1$$

⋮

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1^i + \theta_2 x_2^i + \dots = \sum_{j=0}^n \theta_j x_j^i$$

⋮

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1^m + \theta_2 x_2^m + \dots = \sum_{j=0}^n \theta_j x_j^m$$

$$i = [1, m]$$



Mathematic notations

Now we can transfer the m#'s of linear hypothesis functions into linear algebra:

$$\begin{array}{c} \theta^T \uparrow \end{array} \left\{ \begin{array}{c} \theta_0 \\ \theta_1 \\ \theta_2 \\ \vdots \\ \theta_j \\ \vdots \\ \theta_n \end{array} \right\} \xrightarrow{\quad} \left\{ \begin{array}{c} 1, x^1_1, \dots, x^1_n \\ 1, x^2_1, \dots, x^2_n \\ 1, x^3_1, \dots, x^3_n \\ \vdots \\ 1, x^i_1, \dots, x^i_n \\ \vdots \\ 1, x^m_1, \dots, x^m_n \end{array} \right\} \begin{array}{c} ? \\ \theta \end{array} \xrightarrow{\quad} \text{Most close } h_\theta(x) - y$$

Measure “close”

Aim is to choose $\{\theta_0, \theta_1, \dots\}$ so that $h_\theta(x)$ is close to y for the training examples

Do achieve it, we need a Cost Function/Error Function:

$$J(\theta) = J(\theta_0, \theta_1, \dots, \theta_j) = \frac{1}{2} \sum_{i=1}^m (h_\theta(x^i) - y^i)^2$$

The goal: minimize $J(\theta_0, \theta_1, \dots, \theta_j)$

minimize $J(\theta_0, \theta_1, \theta_2)$

Size in feet ² (x_1)	# Rooms (x_2)	Price (\$) in 1000's (y)
2104	6	460
1416	4	232
1534	5	315
852	3	178
...		...

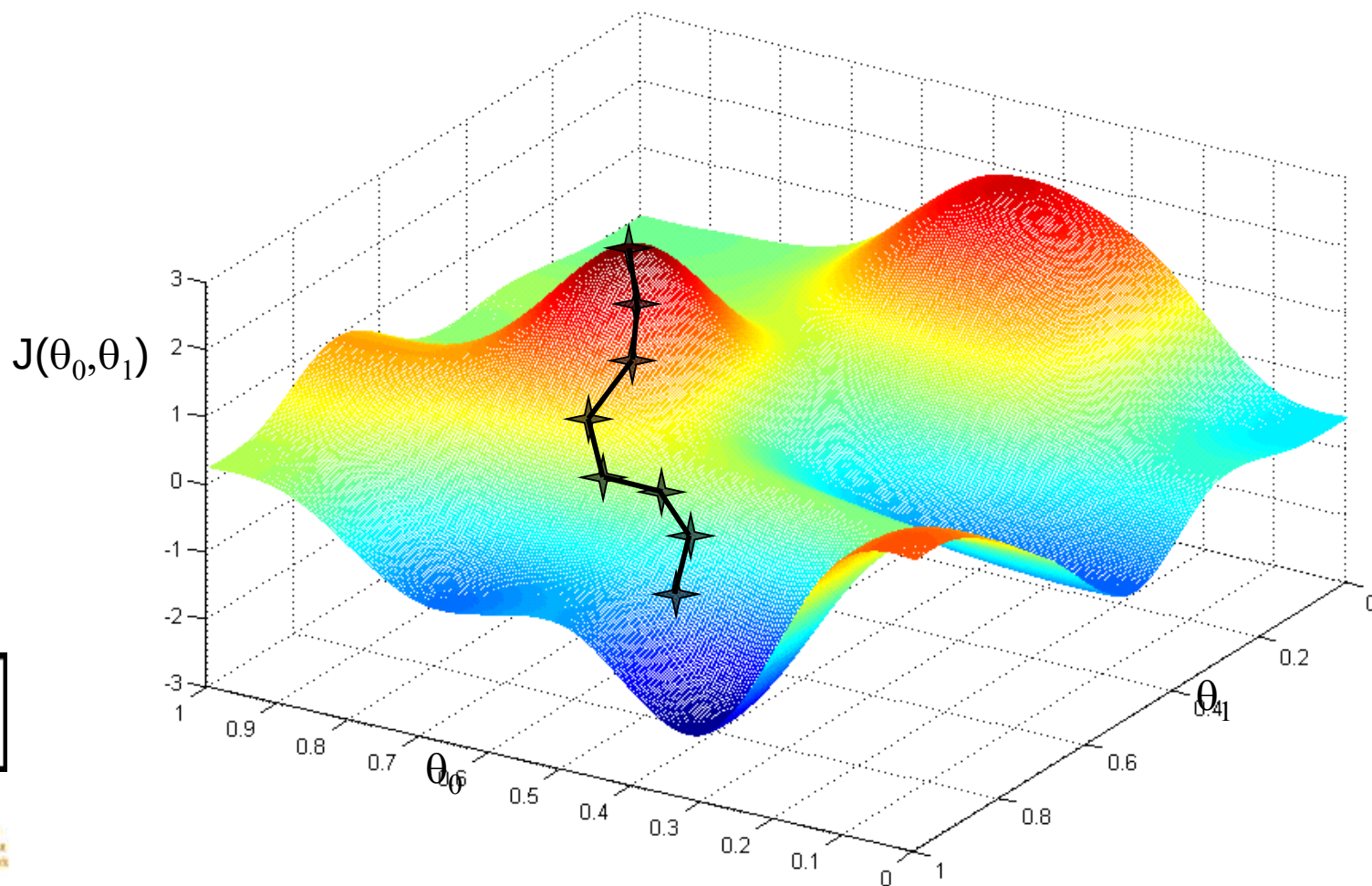
Calculation the closest θ value with the Cost Function

Calculation steps:

1. Start with the initial values of θ
 $\theta_0=0.5, \theta_1=1.2, \theta_2=2.0$ (randomly)
2. Follow a minimizing path to keep changing θ for reducing the cost function J
3. Stop if the J can't be reduced anymore



UNIVERSITY OF
DERBY

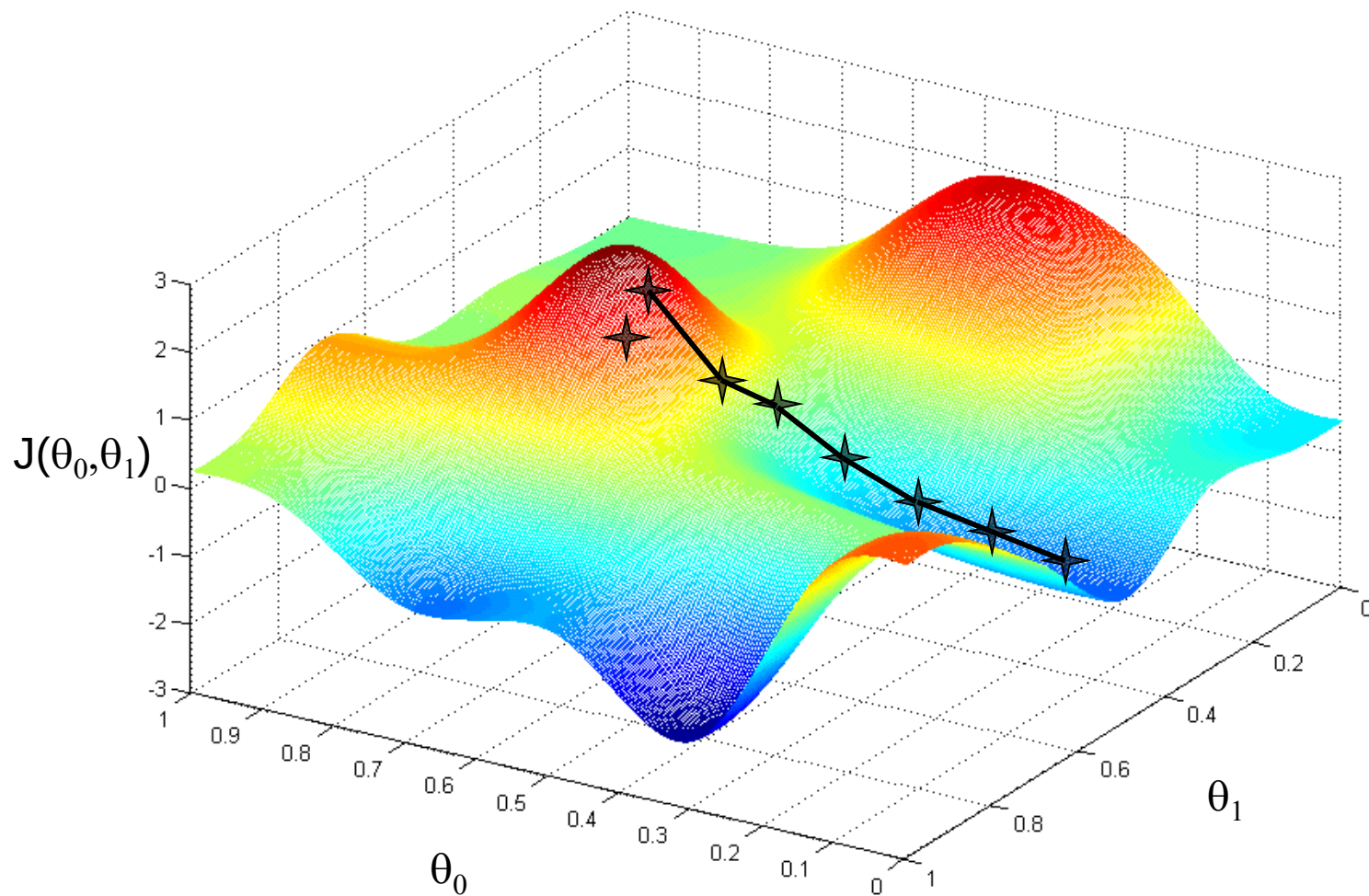


derby.ac.uk

Sensitivity:
Internal



UN
D



IG
UN

TE

derby.ac.uk

LMS (least mean squares)

We want to choose θ so as to minimize $J(\theta)$

Gradient Descent algorithm, let's start with some initial θ , and repeatedly performs the update simultaneously for all θ :

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) \quad | \quad j = 0, \dots, n$$

One training example (x, y)

learning rate

Partial derivative

$$\begin{aligned} \frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \frac{1}{2} (h_{\theta}(x) - y)^2 \\ &= 2 \cdot \frac{1}{2} (h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j} (h_{\theta}(x) - y) \\ &= (h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j} \left(\sum_{i=0}^d \theta_i x_i - y \right) \\ &= (h_{\theta}(x) - y) x_j \end{aligned}$$

Derivatives power rule: $\frac{d}{dx} x^n = nx^{n-1}$

Derivatives chain rule: the derivative of $f(g(x)) = f'(g(x))g'(x)$

For a single training example, this gives the update rule

$$\theta_j := \theta_j + \alpha (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)}$$

Gradient descent algorithm in example

To simplify the explanation and visualisation, we delete the #room feature from the training dataset.

	Size in feet ² (x)	Price (\$) in 1000's (y)
1 ↓ m	2104	460
	1416	232
	1534	315
	852	178
m	...	...

Hypothesis: $h_{\theta}(x) = \theta_0 + \theta_1 x$

Parameters: θ_0, θ_1

Cost Function:

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Goal: minimize $J(\theta_0, \theta_1)$
 θ_0, θ_1

Gradient descent algorithm in code

```
Loop {  
    for  $i = 1$  to  $n$ , {  
  
         $\theta_j := \theta_j + \alpha (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)}$ , (for every  $j$ )  
  
    }  
}
```

Alternative way to minimisation J

Matrix derivatives + Least squares revisited

$$\nabla_A f(A) = \begin{bmatrix} \frac{\partial f}{\partial A_{11}} & \cdots & \frac{\partial f}{\partial A_{1d}} \\ \vdots & \ddots & \vdots \\ \frac{\partial f}{\partial A_{n1}} & \cdots & \frac{\partial f}{\partial A_{nd}} \end{bmatrix}$$

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \nabla_{\theta} \frac{1}{2} (X\theta - \vec{y})^T (X\theta - \vec{y}) \\ &= \frac{1}{2} \nabla_{\theta} ((X\theta)^T X\theta - (X\theta)^T \vec{y} - \vec{y}^T (X\theta) + \vec{y}^T \vec{y}) \\ &= \frac{1}{2} \nabla_{\theta} (\theta^T (X^T X)\theta - \vec{y}^T (X\theta) - \vec{y}^T (X\theta)) \\ &= \frac{1}{2} \nabla_{\theta} (\theta^T (X^T X)\theta - 2(X^T \vec{y})^T \theta) \\ &= \frac{1}{2} (2X^T X\theta - 2X^T \vec{y}) \\ &= X^T X\theta - X^T \vec{y} \end{aligned}$$

Minimize J by explicitly taking its derivatives with respect to the θ_j 's, and setting them to zero

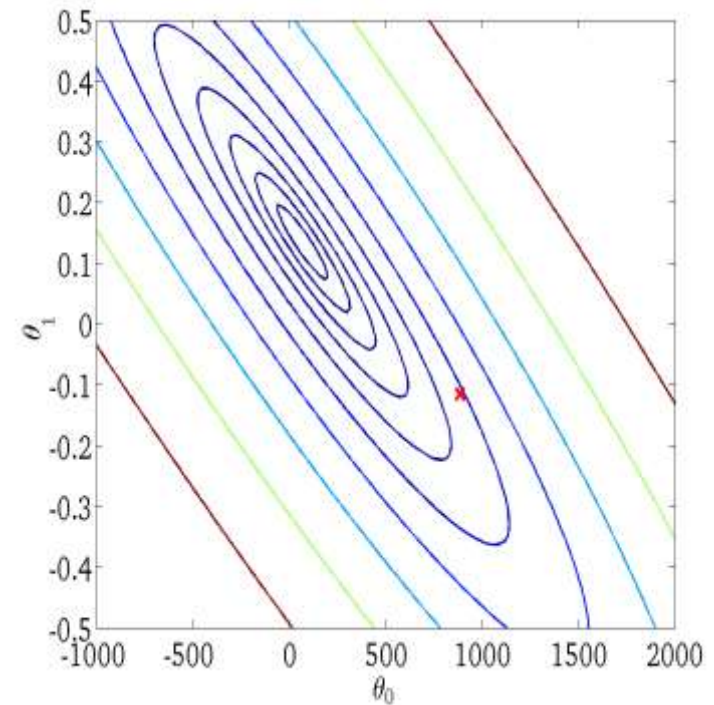
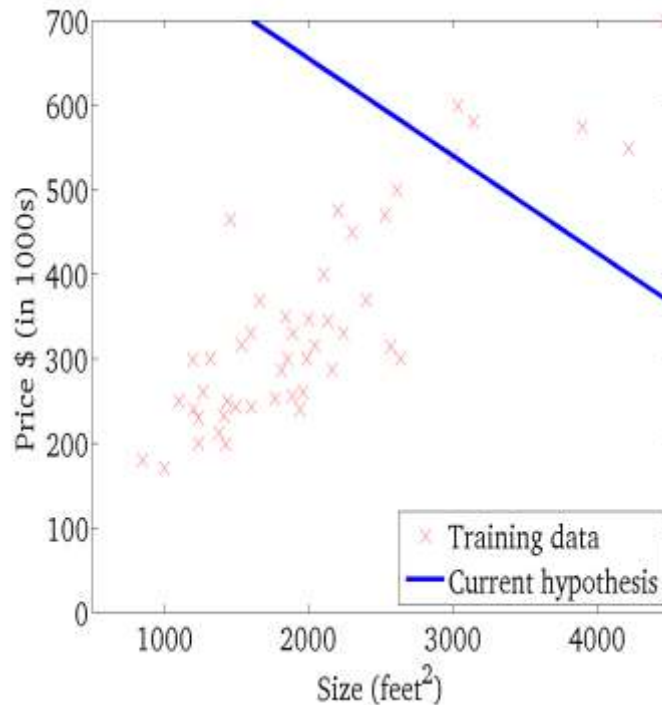
Gradient descent visualisation

$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)

$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)

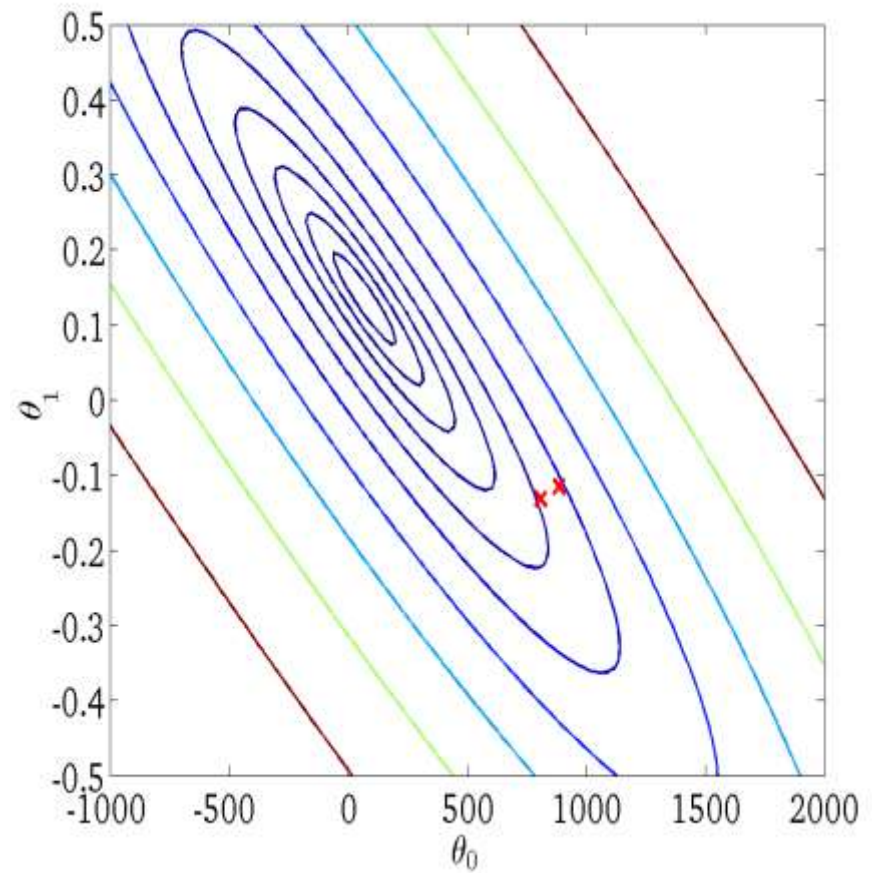
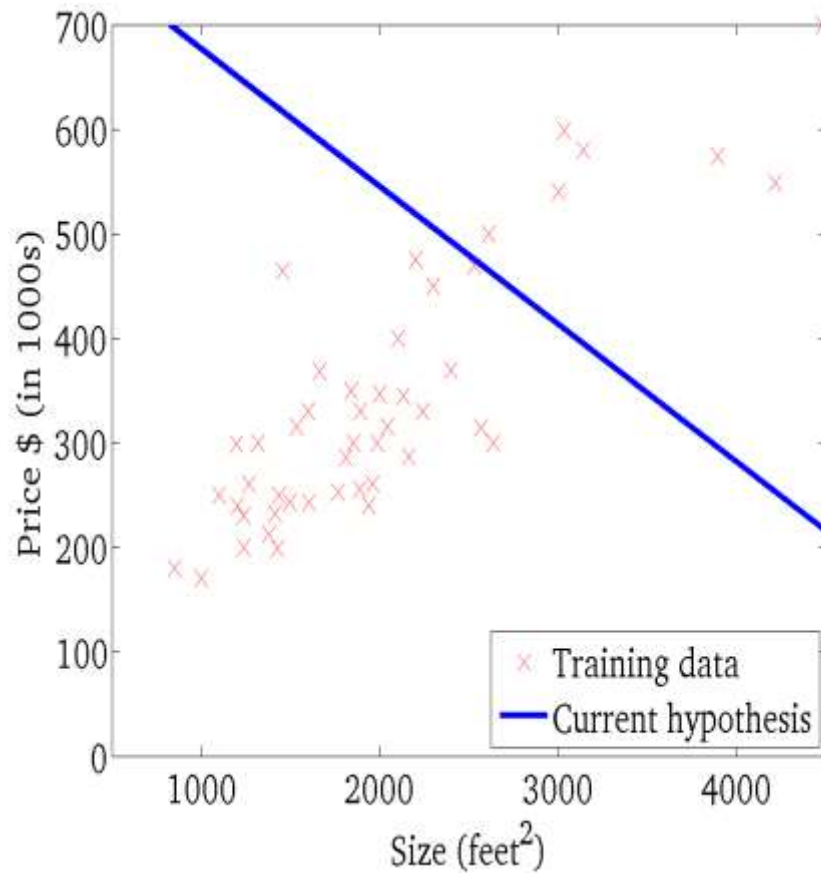


$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)

$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)

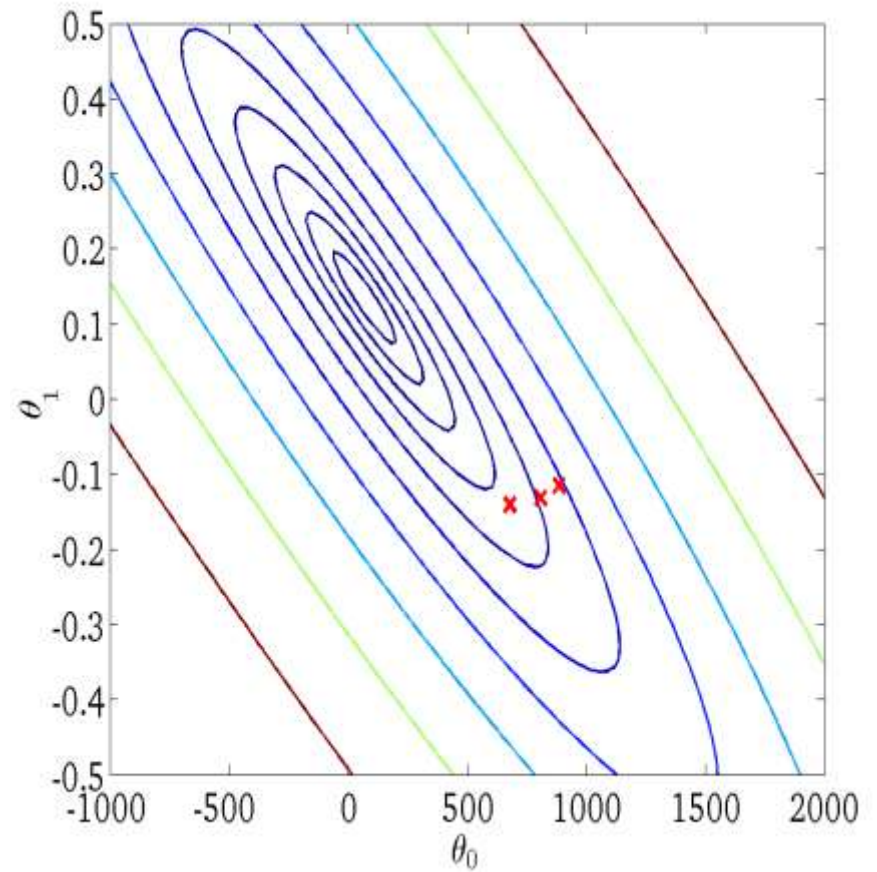
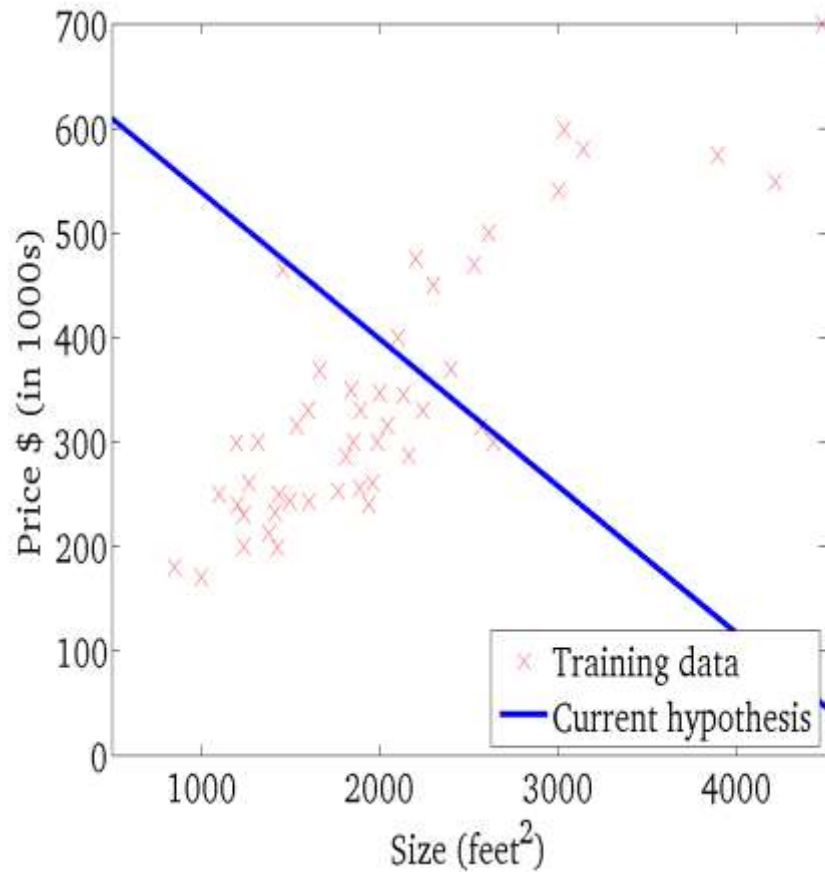


$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)

$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)

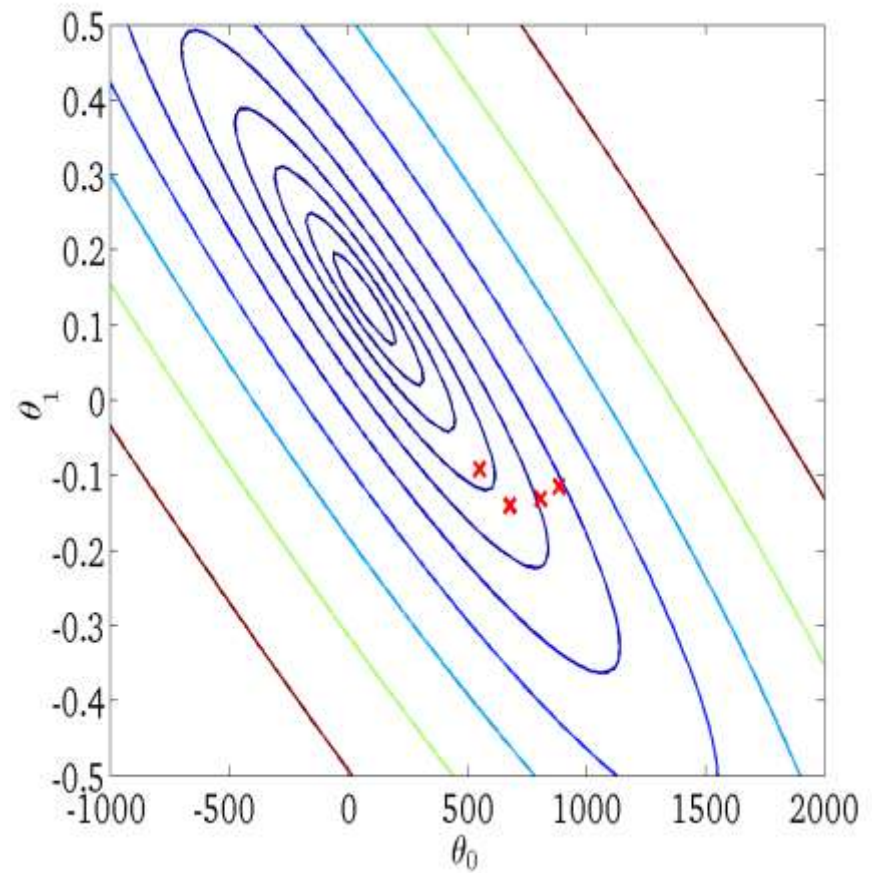
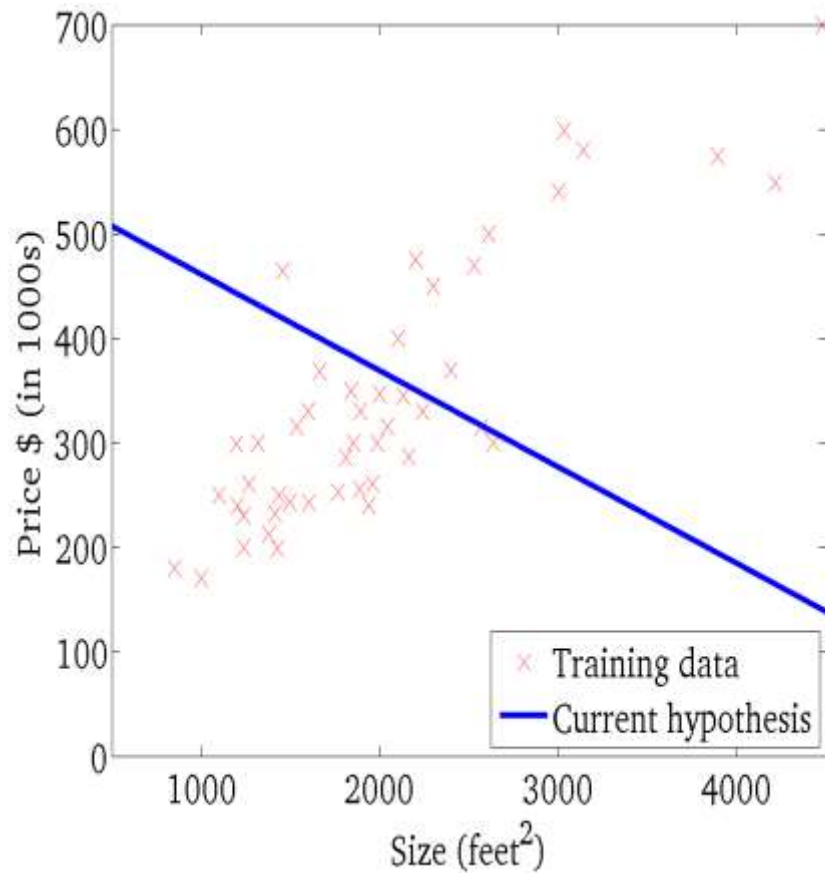


$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)

$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)

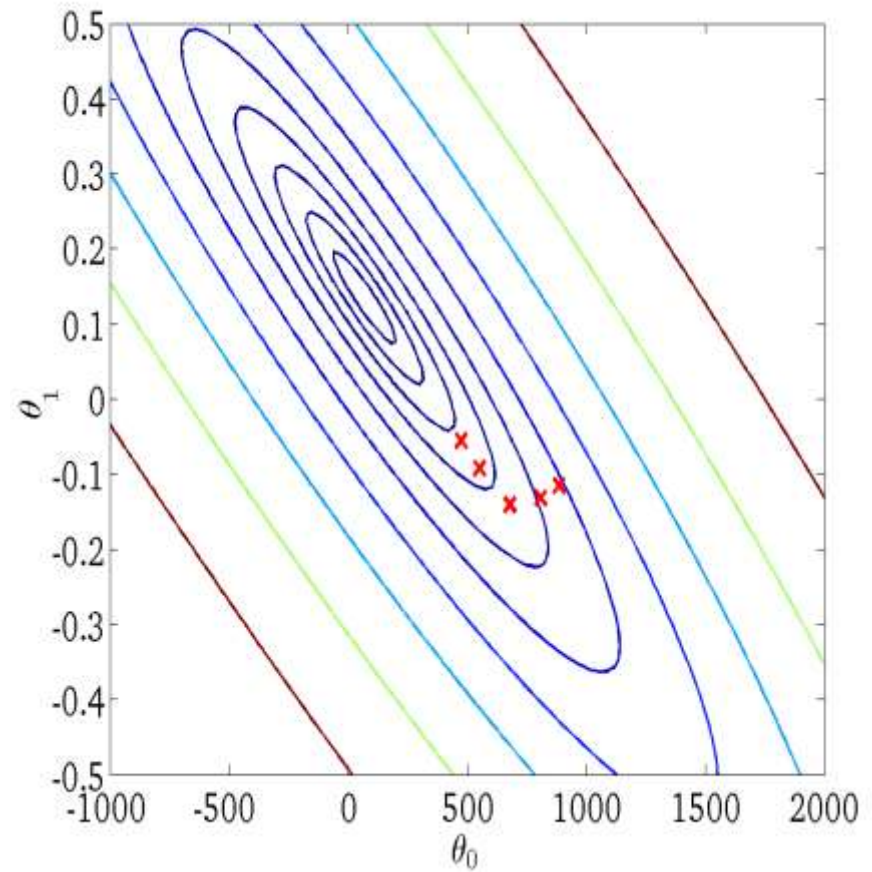
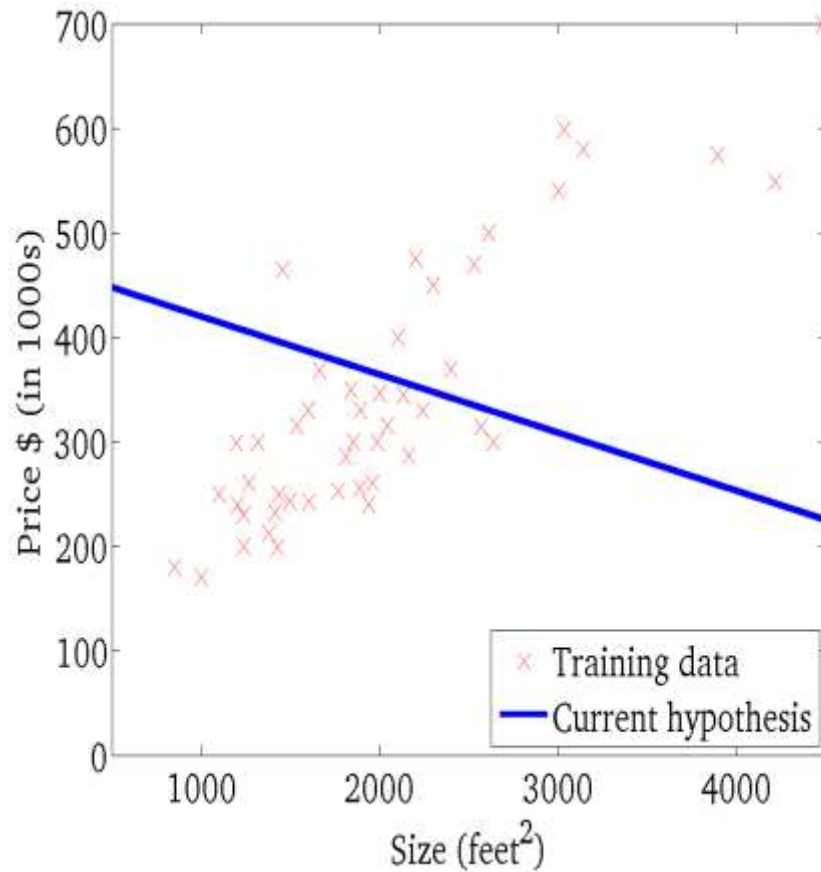


$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)

$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)

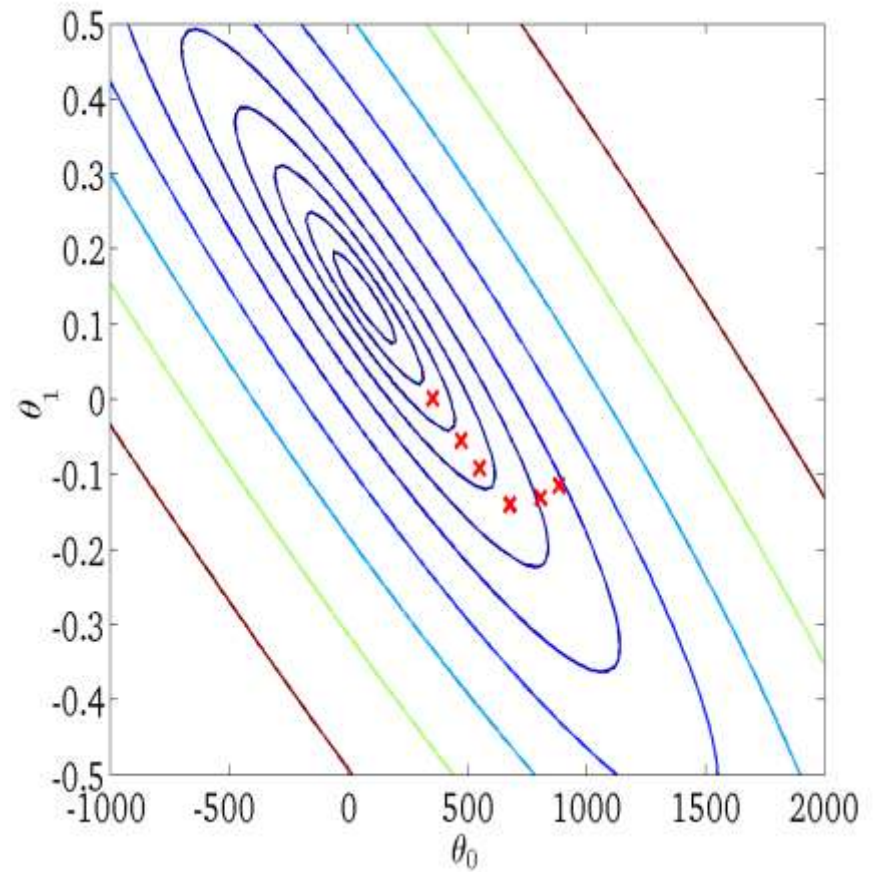
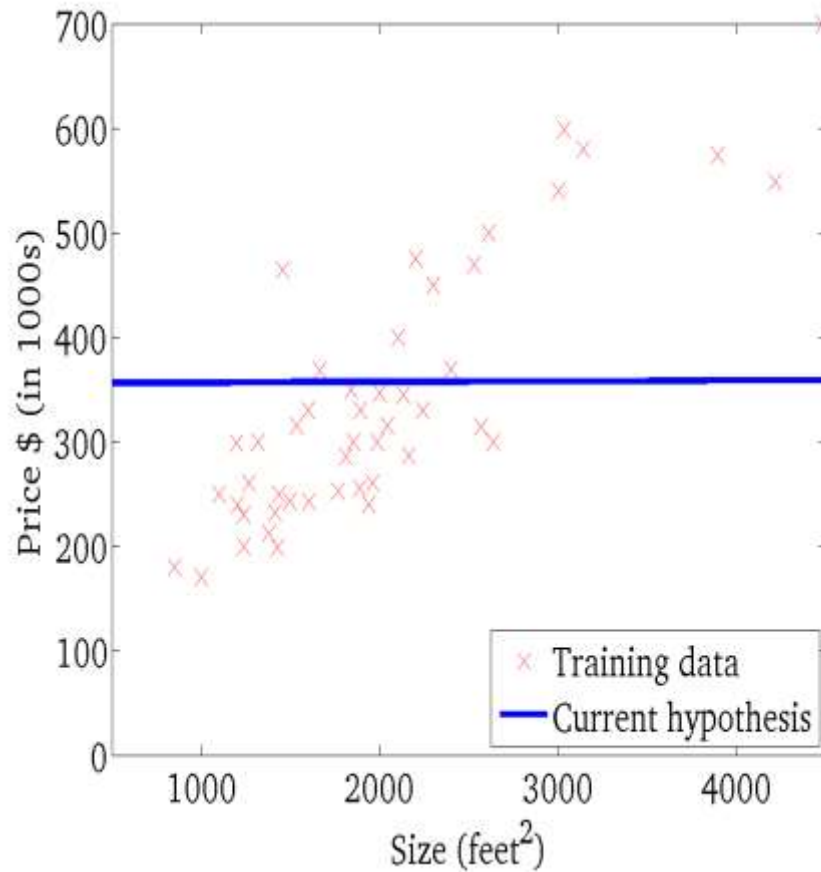


$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)

$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)

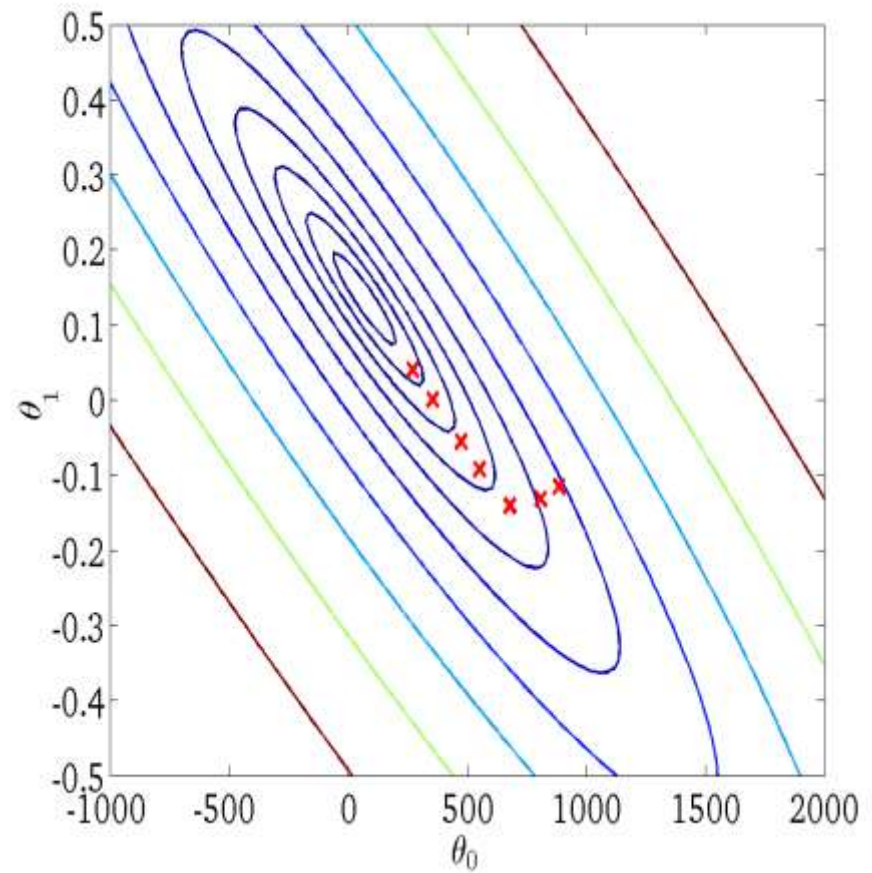
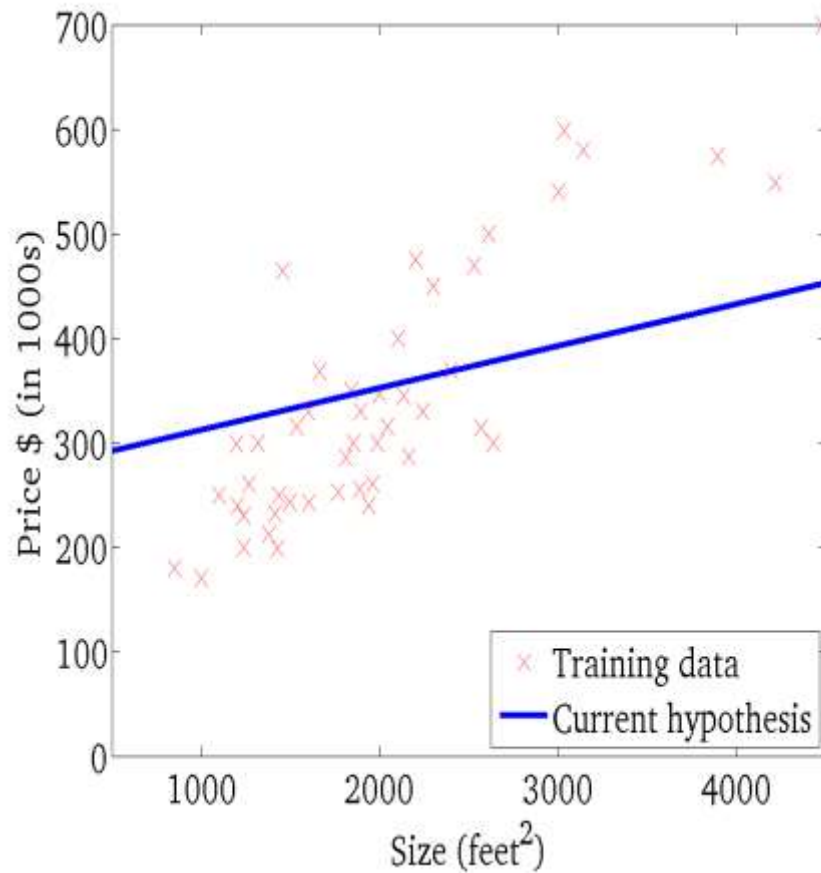


$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)

$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)

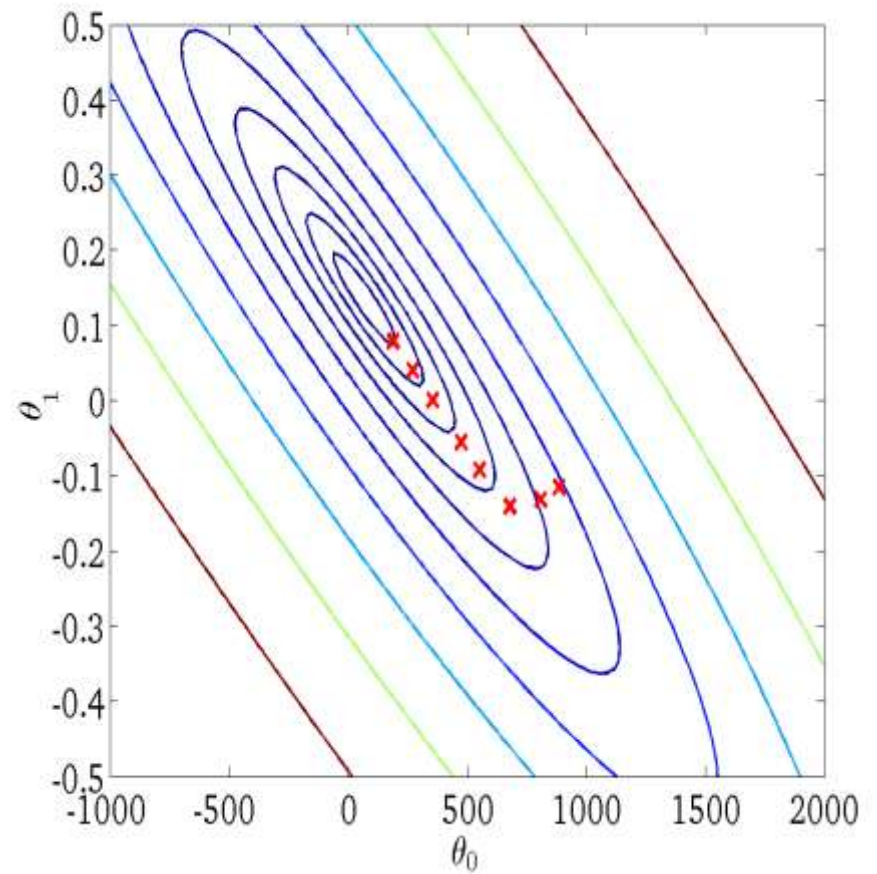
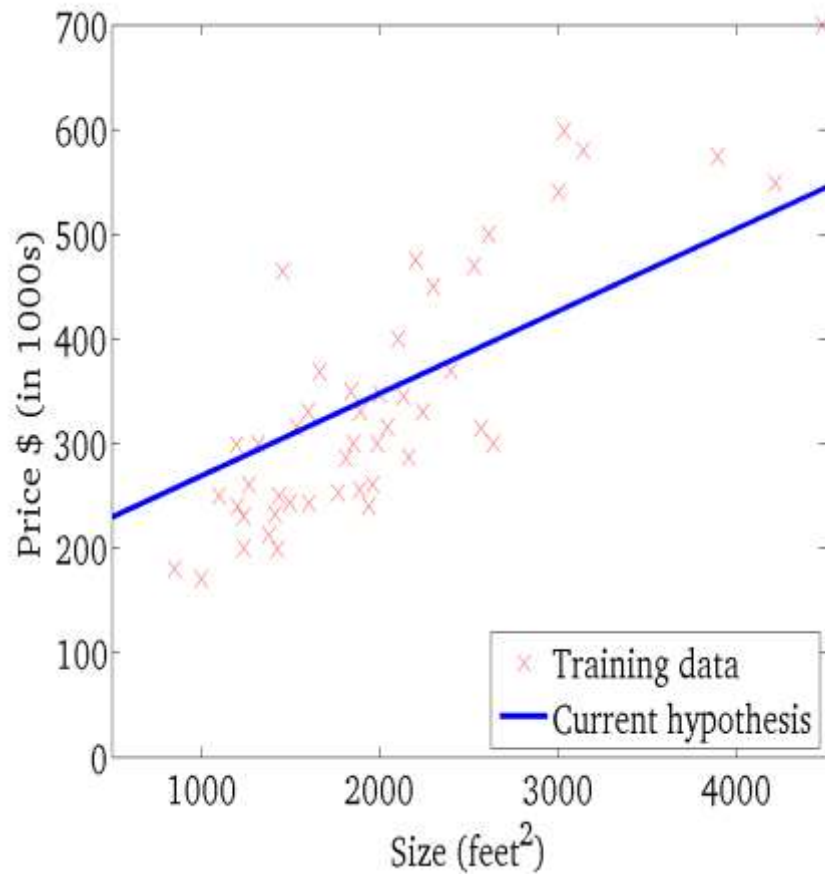


$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)

$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)

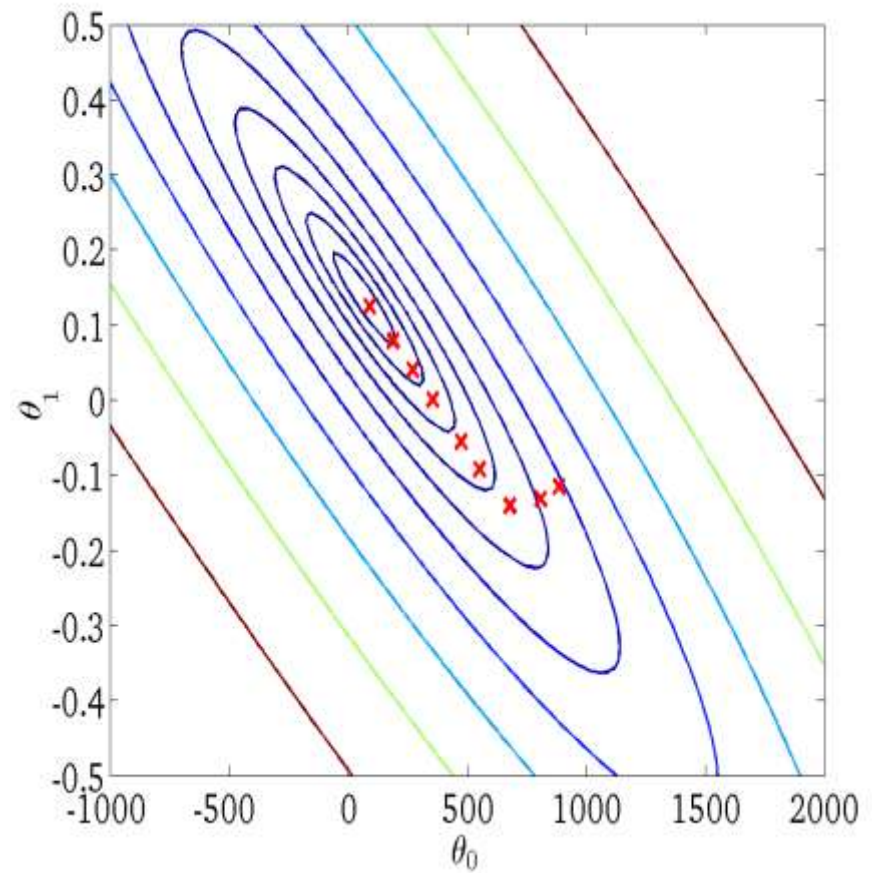
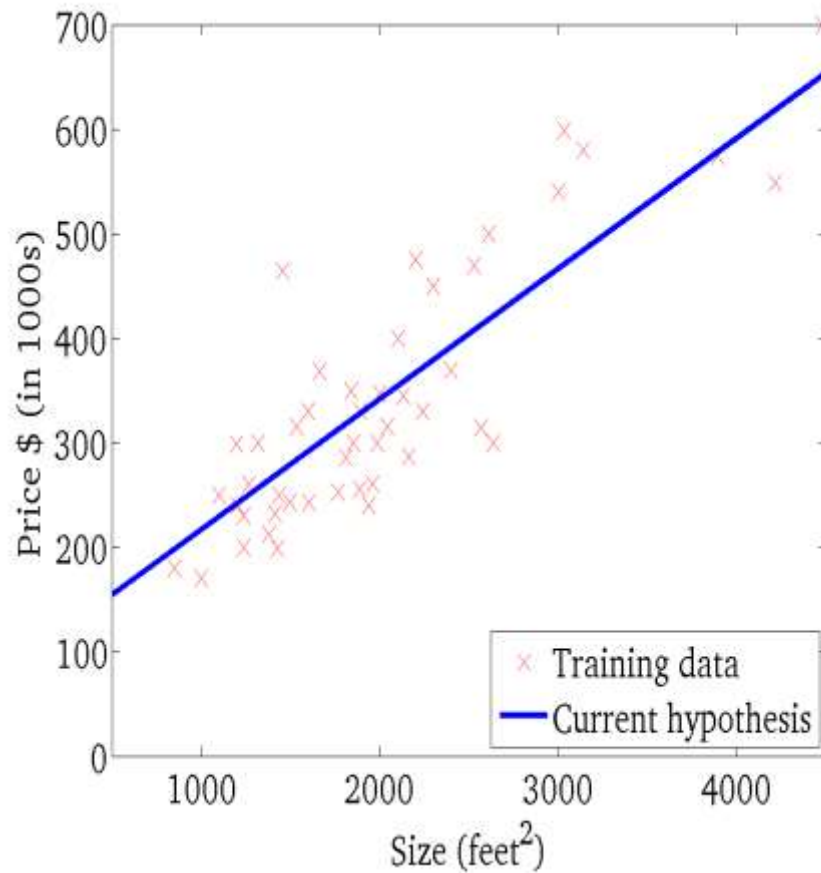


$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)

$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)



“Batch” Gradient Descent

“Batch”: Each step of gradient descent uses all the training examples.

Alternative: process part of the dataset for each step of the algorithm.

The multiple features

	Size (feet ²)	#bedrooms	# floors	Age of home (years)	Price (\$1000)
x_0	x_1	x_2	x_3	x_4	y
1	2104	5	1	45	460
1	1416	3	2	40	232
1	1534	3	2	30	315
1	852	2	1	36	178

n=4

$$X = \begin{bmatrix} 1 & 2104 & 5 & 1 & 45 \\ 1 & 1416 & 3 & 2 & 40 \\ 1 & 1534 & 3 & 2 & 30 \\ 1 & 852 & 2 & 1 & 36 \end{bmatrix}$$

$$y = \begin{bmatrix} 460 \\ 232 \\ 315 \\ 178 \end{bmatrix}$$

Feature Scaling

Idea: Make sure features are on a similar scale.

E.g. x_1 = size (0-2000 feet²)

x_2 = number of bedrooms (1-5)



$$x_1 = \frac{\text{size (feet}^2\text{)}}{2000}$$

$$x_2 = \frac{\text{number of bedrooms}}{5}$$

Get every feature into range

$$-1 \leq x_i \leq 1$$

Feature Scaling

Idea: Make sure features are on a similar scale.

E.g. x_1 = size (0-2000 feet²)

x_2 = number of bedrooms (1-5)



Mean normalization

Replace x_i with $x_i - \mu_i$ to make features

have approximately zero mean
(Do not apply to $x_0 = 1$).

E.g.

$$x_1 = \frac{\text{size} - 1000}{2000}$$

$$-0.5 \leq x_1 \leq 0.5, -0.5 \leq x_2 \leq 0.5$$

$$x_2 = \frac{\#bedrooms - 2}{5}$$

Gradient descent

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

- “Debugging”: How to make sure gradient descent is working correctly.
- How to choose learning rate α .

Choosing α

- If α is too small: slow convergence.
- If α is too large: $J(\theta)$ may not decrease on every iteration; may not converge.

To choose α , try

$\dots, 0.001, \quad , 0.01, \quad , 0.1, \quad , 1, \dots$

Outlook

1. Linear regression

- Examples
- Math notation
- Least Mean Square function (LMS)
- Gradient Descent function

2. Logistic regression

Logistic Regression

Logistic Regression (LR) is a model (algorithm) used to solve a classification problem.

Especially, LR aims to binary classification problem that means the target class $y = \{0,1\}$.

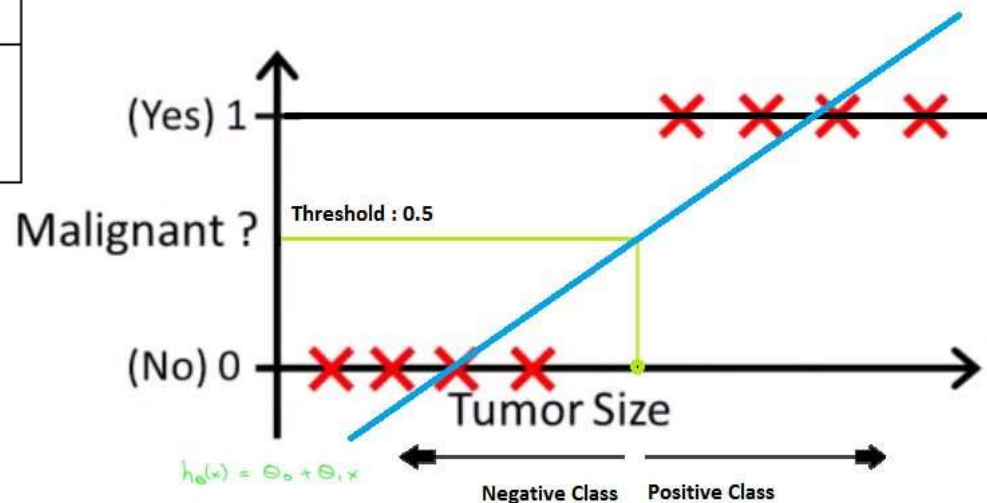
The reason LR is still a regression model because it's underlying techniques is very close to Linear Regression.

The term logistic is taken purely because the Logit Function is used to do the classification.

Binary Classification problem examples

	Two Class Classification	
$y \in \{0, 1\}$	1 or Positive Class	0 or Negative Class
Email	Spam	Not Spam
Tumor	Malignant	Benign
Transaction	Fraudulent	Not Fraudulent

Why cannot use Linear Regression?



Logistic Regression model

The logistic function is a Sigmoid function, which takes any real value between zero and one. So the logistic regression hypothesis is defined as:

$$h_{\theta}(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}},$$

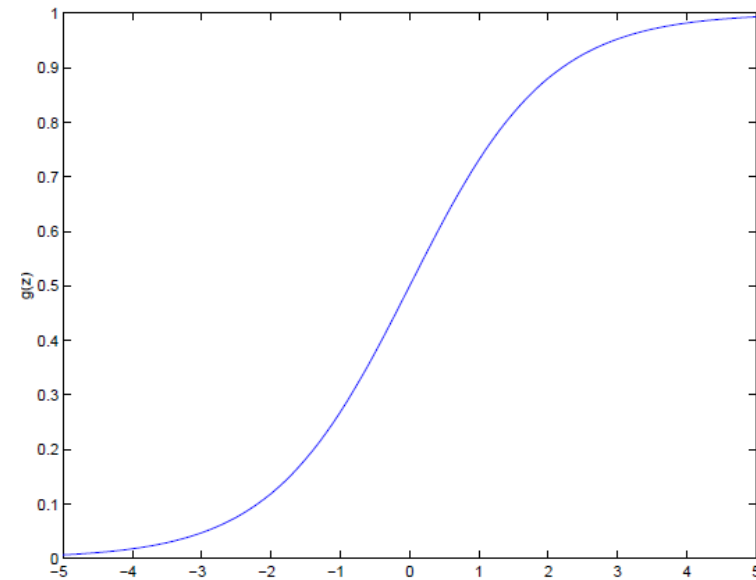
where

$$g(z) = \frac{1}{1 + e^{-z}}$$

$g(z)$ tends towards 1 as $z \rightarrow \infty$

$g(z)$ tends towards 0 as $z \rightarrow -\infty$.


$$\theta^T x = \theta_0 + \sum_{j=1}^d \theta_j x_j$$



Important property

Before moving on, here's a useful property of the derivative of the sigmoid function, which we write as g' :

$$\begin{aligned}
 g'(z) &= \frac{d}{dz} \frac{1}{1 + e^{-z}} \\
 &= \frac{1}{(1 + e^{-z})^2} (e^{-z}) \\
 &= \frac{1}{(1 + e^{-z})} \cdot \left(1 - \frac{1}{(1 + e^{-z})}\right) \\
 &= g(z)(1 - g(z)).
 \end{aligned}$$

$Z=X$

$$= \frac{1 \cdot e^{-x}}{(1 + e^{-x}) \cdot (1 + e^{-x})} = \frac{1}{(1 + e^{-x})} \cdot \frac{e^{-x}}{(1 + e^{-x})}$$

And since $+1 - 1 = 0$ we can do this

$$= \frac{1}{(1 + e^{-x})} \cdot \frac{e^{-x}}{(1 + e^{-x})} = \frac{1}{(1 + e^{-x})} \cdot \frac{e^{-x} + 1 - 1}{(1 + e^{-x})}$$

And now let's break the fraction and rewrite it as

$$= \frac{1}{(1 + e^{-x})} \cdot \frac{e^{-x} + 1 - 1}{(1 + e^{-x})} = \frac{1}{(1 + e^{-x})} \cdot \left(\frac{1 + e^{-x}}{1 + e^{-x}} - \frac{1}{1 + e^{-x}}\right)$$

Let's cancel out the numerator and denominator

$$= \frac{1}{(1 + e^{-x})} \cdot \left(\frac{1 + e^{-x}}{1 + e^{-x}} - \frac{1}{1 + e^{-x}}\right) = \frac{1}{(1 + e^{-x})} \cdot \left(1 - \frac{1}{1 + e^{-x}}\right)$$

Internal

Reciprocal rule: $\left[\frac{1}{u(x)}\right]' = [u(x)^{-1}]' = -\left[\frac{u'(x)}{u(x)^2}\right] = -u(x)^{-2} \cdot u'(x)$

$$= \frac{d}{dx} (1 + e^{-x})^{-1} = -(1 + e^{-x})^{-2} \cdot \frac{d}{dx} (1 + e^{-x})$$

Linearity rule: $[a \cdot u(x) + b \cdot v(x)]' = a \cdot u'(x) + b \cdot v'(x)$

$$= -(1 + e^{-x})^{-2} \cdot \left(\frac{d}{dx} [1] + \frac{d}{dx} [e^{-x}]\right) = -(1 + e^{-x})^{-2} \cdot \left(\frac{d}{dx} [e^{-x}]\right)$$

Exponential rule: $[e^{u(x)}]' = e^{u(x)} \cdot u'(x)$

$$= -(1 + e^{-x})^{-2} \cdot (e^{-x} \cdot \frac{d}{dx} [-x])$$

Linearity rule: $[a \cdot u(x) + b \cdot v(x)]' = a \cdot u'(x) + b \cdot v'(x)$

$$= -(1 + e^{-x})^{-2} \cdot (e^{-x} \cdot \frac{d}{dx} [-x]) = -(1 + e^{-x})^{-2} \cdot (e^{-x} \cdot -\frac{d}{dx} [x])$$

$$= -(1 + e^{-x})^{-2} \cdot (e^{-x} \cdot -1)$$

$$= (1 + e^{-x})^{-2} \cdot e^{-x}$$

Search for θ

Given the logistic regression model, how do we fit θ for it?

$$h_{\theta}(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}},$$

where

$$g(z) = \frac{1}{1 + e^{-z}}$$

$$\theta^T x = \theta_0 + \sum_{j=1}^d \theta_j x_j$$

Probabilistic assumptions

$$P(y = 1 \mid x; \theta) = h_{\theta}(x)$$

$$P(y = 0 \mid x; \theta) = 1 - h_{\theta}(x)$$



$$p(y \mid x; \theta) = (h_{\theta}(x))^y (1 - h_{\theta}(x))^{1-y}$$



$$L(\theta) = p(\vec{y} \mid X; \theta)$$



$$= \prod_{i=1}^n p(y^{(i)} \mid x^{(i)}; \theta)$$

$$= \prod_{i=1}^n (h_{\theta}(x^{(i)}))^{y^{(i)}} (1 - h_{\theta}(x^{(i)}))^{1-y^{(i)}}$$

$$\ell(\theta) = \log L(\theta)$$

$$= \sum_{i=1}^n y^{(i)} \log h(x^{(i)}) + (1 - y^{(i)}) \log(1 - h(x^{(i)}))$$

Search for θ (cont.)

Gradient ascent rule:

$$\begin{aligned}
 \frac{\partial}{\partial \theta_j} \ell(\theta) &= \left(y \frac{1}{g(\theta^T x)} - (1 - y) \frac{1}{1 - g(\theta^T x)} \right) \frac{\partial}{\partial \theta_j} g(\theta^T x) \\
 &= \left(y \frac{1}{g(\theta^T x)} - (1 - y) \frac{1}{1 - g(\theta^T x)} \right) g(\theta^T x)(1 - g(\theta^T x)) \frac{\partial}{\partial \theta_j} \theta^T x \\
 &= (y(1 - g(\theta^T x)) - (1 - y)g(\theta^T x)) x_j \\
 &= (y - h_{\theta}(x)) x_j
 \end{aligned}$$

$$\begin{aligned}
 g'(z) &= \frac{d}{dz} \frac{1}{1 + e^{-z}} \\
 &= \frac{1}{(1 + e^{-z})^2} (e^{-z}) \\
 &= \frac{1}{(1 + e^{-z})} \cdot \left(1 - \frac{1}{(1 + e^{-z})} \right) \\
 &= g(z)(1 - g(z)).
 \end{aligned}$$

$$\theta_j := \theta_j + \alpha (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)}$$

The identical update rule for linear regression

Logistic Regression Python

Train Test Split

+ Code

+ Markdown

```
1: from sklearn.model_selection import train_test_split
```

```
1: X_train, X_test, y_train, y_test = train_test_split(train.drop('Survived', axis=1),  
                                                    train['Survived'], test_size=0.30,  
                                                    random_state=101)
```

Training and Predicting

```
1: from sklearn.linear_model import LogisticRegression
```

```
2: logmodel = LogisticRegression()  
   logmodel.fit(X_train, y_train)
```

Evaluation

```
predictions = logmodel.predict(X_test)
```

Let's move on to evaluate our model!

Evaluation

+ Code

+ Markdown

We can check precision, recall, f1-score using classification report!

```
from sklearn.metrics import classification_report
```

```
print(classification_report(y_test, predictions))
```

	precision	recall	f1-score	support
0	0.79	0.91	0.85	163
1	0.81	0.62	0.71	104
accuracy			0.80	267
macro avg	0.80	0.77	0.78	267
weighted avg	0.80	0.80	0.79	267

Summary

The linear regression (definition and examples)

Linear and Logistic regression process and math foundations

- Math notations, LMS, Gradient Descent algorithm

Make sure your linear regression works correctly

- Scaling (normalisation), and select learning rate